NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

ATTRIBUTE GRAMMARS

IN OFLAT

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: February 1, 2025

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

ATTRIBUTE GRAMMARS

IN OFLAT

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

Adviser: Artur Miguel Dias
Professor Auxiliar, DI-FCT, Universidade NOVA de Lisboa

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: *February 1, 2025*

Abstract

Theoretical concepts in Formal Languages and Automata Theory (FLAT) are known for their formal, rigorous, and often complex nature, making them challenging to learn. To address this challenge, various tools have been developed over the years, allowing users to engage with these concepts through practical exercises and examples.

At FCT-UNL, two pedagogical tools were created to aid in this learning process: the OCaml-FLAT library, which provides logical support for FLAT concepts, and the OFLAT graphical application, a web-based tool offering interactive visualizations of these ideas.

The first goal of this dissertation is to extend the OCaml-FLAT library by incorporating support for attribute grammars. The second goal is to enhance the OFLAT graphical application by adding features that allow for the visualization and interaction with these grammar concepts.

A key focus of this work is the pedagogical enhancement of the developed software. Consequently, a significant portion of the dissertation will be devoted to explaining the theoretical foundations of attribute grammars, including various classic variants.

Keywords: Formal Languages, Automata Theory, OCaml, OFLAT, OCaml-FLAT, Functional Programming, Formal Grammars, Pedagogical Tools

Resumo

Os conceitos teóricos em Teoria de Linguagens Formais e Autómatos (FLAT) são conhecidos pela sua natureza formal, rigorosa e muitas vezes complexa, o que os torna desafiantes para aprender. Para mitigar essa dificuldade, várias ferramentas foram desenvolvidas ao longo dos anos, permitindo que os utilizadores interajam com esses conceitos através de exercícios e exemplos práticos.

Na FCT-UNL, foram criadas duas ferramentas pedagógicas com este propósito: a biblioteca OCaml-FLAT, que fornece suporte lógico para os conceitos de FLAT, e a aplicação gráfica OFLAT, uma ferramenta web que oferece visualizações interativas desses conceitos.

O primeiro objetivo desta dissertação é estender a biblioteca OCaml-FLAT, incorporando suporte para gramáticas com atributos. O segundo objetivo é melhorar a aplicação gráfica OFLAT, adicionando funcionalidades que permitam a visualização e interatividade com estes conceitos de gramáticas.

Um foco importante deste trabalho é o reforço do caráter pedagógico do software a ser desenvolvido. Assim, uma parte significativa da dissertação será dedicada à explicação dos fundamentos teóricos das gramáticas com atributos, incluindo as suas diferentes variantes clássicas.

Palavras-chave: Teoria de Linguagens Formais, Autómatos, OCaml, OFLAT, OCaml-FLAT, Programação Funcional, Gramáticas Formais, Ferramentas Pedagógicas

Contents

List of Figures **v**

1 Introduction **1**
1.1 Context . . . 1
1.2 Objectives . . . 1
1.3 Document Structure . . . 1

2 Theory of Computation **2**
2.1 Chomsky Hierarchy . . . 2
2.1.1 Type 0: Unrestricted Grammars . . . 2
2.1.2 Type 1: Context-Sensitive Grammars . . . 3
2.1.3 Type 2: Context-Free Grammars . . . 3
2.1.4 Type 3: Regular Grammars . . . 3
2.2 Context-Free Grammars . . . 4
2.3 Attribute Grammars . . . 4
2.3.1 Synthesized Attributes . . . 5
2.3.2 Inherited Attributes . . . 6
2.3.3 Circularity . . . 6
2.4 Attribute Grammars with Constraints . . . 7
2.4.1 Example Grammar . . . 7
2.4.2 Synthesized Attribute: Length . . . 7
2.4.3 Inherited Attribute: Permission . . . 7
2.4.4 Defining the Language . . . 8
2.4.5 Applications . . . 8
2.5 Automata . . . 8
2.5.1 Types of Automata . . . 8
2.5.2 The Relationship Between Automata and Formal Languages . . . 9
2.5.3 Applications of Automata and Formal Languages . . . 10

3 Grammar Tools **11**

3.1 JFLAT . . . 11
3.1.1 Features . . . 11
3.1.2 Educational Benefits . . . 12
3.1.3 Case Studies/Examples . . . 12
3.2 ANTLR . . . 12
3.2.1 Features . . . 12
3.2.2 Educational Benefits . . . 13
3.2.3 Case Studies/Examples . . . 13
3.3 CGM: Context-Free Grammar Manipulator . . . 13
3.3.1 Features . . . 13
3.3.2 Educational Benefits . . . 14
3.3.3 Case Studies/Examples . . . 14
3.4 Comparison of Tools . . . 14
3.4.1 Features . . . 14
3.4.2 Ease of Use . . . 15
3.4.3 Educational Impact . . . 15
3.4.4 Strengths and Weaknesses . . . 15

4 OCaml-Flat and OFLAT Systems **17**
4.1 OCamlFLAT . . . 17
4.2 OFLAT Platform . . . 18

5 Technology **19**
5.1 Declarative Code in OCaml . . . 19
5.2 Functional Declarative Programming . . . 19
5.3 OCaml/JavaScript Interoperability . . . 20
5.3.1 JavaScript Types . . . 20
5.3.2 Bindings . . . 20
5.3.3 `js_of_ocaml` Modules . . . 21
5.4 Representation in OCaml . . . 21
5.4.1 Synthesized Attributes . . . 21
5.4.2 Inherited Attributes . . . 21
5.4.3 Rules . . . 22
5.5 Cytoscape . . . 22

6 Lorem Ipsum **23**

List of Figures

1

Introduction

1.1 Context

The study of Formal Languages and Automata Theory (FLAT) is integral to Computer Engineering and Computer Science programs. However, due to its abstract and mathematical nature, students often struggle to fully grasp the concepts and models involved. Therefore, a tool that complements the study of FLAT theory by being practical, user-friendly, and visually oriented is highly beneficial.

In recent years, the OCaml-FLAT/OFLAT tool has been developed at FCT-UNL, offering extensive support in this field. Despite its advanced state, with support for several key models, there are still many features that could enhance its functionality.

This dissertation aims to introduce new capabilities to OCamlFLAT/OFLAT, enhancing both the OCamlFLAT library and the OFLAT graphical interface.

1.2 Objectives

The primary objective of this dissertation is to incorporate support for attribute grammars. Attribute grammars extend context-free grammars by allowing additional information (attributes) to be associated with production rules. These attributes are crucial for describing the semantic properties of the languages generated by the grammars.

To achieve this, the OCamlFLAT library needs to be extended to support attribute grammars. Additionally, graphical enhancements are required to enable the creation and editing of these grammars within the OFLAT platform.

1.3 Document Structure

2

Theory of Computation

In Theory of Computation, various models of computation with different levels of expressive power are studied. To discuss and compare different levels of expressive power, it is inevitable to consider algorithms in the context of some application domain. The typically chosen application domain is the syntax of formal languages. There are two techniques for defining languages that are typically used. The first technique is based on the concept of generation and uses the formalism of grammars. Another technique is based on the mechanism of recognition and uses the formalism of automata.

2.1 Chomsky Hierarchy

Noam Chomsky (with the help of Marcel-Paul Schützenberger) identified several classes of formal languages, with different levels of specification complexity, requiring computational models with different levels of expressive power to solve the problem of recognizing these languages.

There are four levels: type 0, type 1, type 2, and type 3.

- Type 0 — Unrestricted Grammars
- Type 1 — Context-Sensitive languages
- Type 2 — Context-Free languages
- Type 3 — Regular languages

The Chomsky hierarchy involves four levels of language types. Each type of language is associated with a type of generator and a type of recognizer.

2.1.1 Type 0: Unrestricted Grammars

Type 0 grammars, also known as unrestricted grammars, are the most general class in the Chomsky hierarchy. They have no restrictions on their production rules, meaning that any string of symbols can be replaced by any other string of symbols. This flexibility allows

Type 0 grammars to generate any language that is recognizable by a Turing machine, making them equivalent in power to Turing machines. In other words, they can describe all recursively enumerable languages, which are the most extensive class of languages in formal language theory.

An example of an unrestricted grammar is:

$$G = (\{S, A, B\}, \{a, b\}, P, S)$$

$$P = \{S \to aAB, AB \to BA, A \to a, B \to b\}$$

2.1.2 Type 1: Context-Sensitive Grammars

Type 1 grammars, or context-sensitive grammars, these grammars do not reduce the length of the sentential form, meaning that the length of the string on the left-hand side of a production rule is less than or equal to the length of the string on the right-hand side. This property ensures that the derivation process does not shorten the string, maintaining context sensitivity. Type 1 grammars can generate languages that are recognized by a linear bounded automaton, which is a non-deterministic Turing machine with a tape of limited length proportional to the input size.

An example of a context-sensitive grammar is:

$$G = (\{S, A, B, C\}, \{a, b, c\}, P, S)$$

$$P = \{S \to aABC, AB \to BA, BC \to CB, aA \to aa, bB \to bb, cC \to cc\}$$

2.1.3 Type 2: Context-Free Grammars

Type 2 grammars, or context-free grammars, in these grammars have a specific form: the left-hand side consists of a single non-terminal symbol, and the right-hand side can be any string of terminal and/or non-terminal symbols. This structure allows context-free grammars to generate languages that can be recognized by a pushdown automaton, which is a type of automaton that uses a stack to manage additional information.

An example of a context-free grammar is:

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = \{S \to aSb, S \to ab\}$$

2.1.4 Type 3: Regular Grammars

Type 3 grammars, also known as regular grammars, represent the most constrained category within the Chomsky hierarchy. The production rules for these grammars are quite specific: each rule's left-hand side is a single non-terminal symbol, while the right-hand side is either a single terminal symbol or a terminal symbol followed by a single

non-terminal symbol. This strict format allows regular grammars to generate languages that can be processed by finite automata, which are basic computational models with limited memory capacity.

An example of a regular grammar is:

$$G = (\{S\}, \{a, b\}, P, S)$$
$$P = \{S \rightarrow aS, S \rightarrow b\}$$

For Type 3 grammars, it is important to understand the concepts of generator and recognizer. A generator is a grammar that defines a language, while a recognizer is an automaton that accepts or rejects strings of that language.

This dissertation will focus on type 2, context-free grammars.

2.2 Context-Free Grammars

Context-free grammars (CFGs) are a special type of formal grammar where all production rules are of the form $A \rightarrow \alpha$, where A is a non-terminal symbol and α is a string of terminals and/or non-terminals. CFGs are fundamental in the field of formal language theory and are widely used in the description of programming languages and in syntactic analysis.

These grammars are used to define the syntax of programming languages, such as C, Java, and Python. In compiler design, they help parse source code and check for syntactic correctness. Additionally, CFGs model the syntax of natural languages, aiding in tasks like sentence parsing and grammar checking.

CFGs are crucial in the development of parsers, which are components of compilers that analyze the structure of code. Efficient parsing algorithms, such as LL and LR parsers, are designed to work with CFGs, making them indispensable in the field of compiler construction.

Example:

$$G = (\{S, A, B\}, \{a, b\}, P, S)$$
$$P = \{S \rightarrow aSb, S \rightarrow ab\}$$

This grammar generates the language of all strings that have the same number of a's and b's.

2.3 Attribute Grammars

Attribute grammars are an extension of context-free grammars that associate attributes with the grammar symbols and define rules for computing these attributes. These attributes provide a way to carry semantic information through the parse tree, enabling context-sensitive analysis.

There are two main types of attributes:

- **Synthesized Attributes**: These are computed from the attributes of the children nodes in the parse tree. They propagate information up the tree.
- **Inherited Attributes**: These are computed from the attributes of the parent and/or sibling nodes. They propagate information down and across the tree.

Attribute grammars are used in various applications, such as:

- **Compiler Design**: They help in semantic analysis, type checking, and intermediate code generation.
- **Language Translation**: They facilitate the translation of high-level programming languages into machine code or intermediate representations.
- **Natural Language Processing**: They assist in understanding and processing the syntactic and semantic structure of natural languages.

By extending context-free grammars with attributes, attribute grammars enable more powerful and flexible language descriptions, making them essential tools in the field of computer science

2.3.1 Synthesized Attributes

Synthesized attributes are computed from the attributes of the children of a node in the derivation tree. These attributes propagate information up the tree, allowing for the evaluation of properties that depend on the structure of the subtree rooted at a particular node.

For example, consider the following grammar:

$$G = (\{S, A\}, \{a, b\}, P, S)$$
$$P = \{S \to A, A \to aA|b\}$$

We can define a synthesized attribute length that calculates the length of the generated string. The rules for computing this attribute are as follows:

$$\text{length}(S) = \text{length}(A)$$
$$\text{length}(A \to aA) = 1 + \text{length}(A)$$
$$\text{length}(A \to b) = 1$$

In this example, the attribute length is synthesized because it is computed based on the attributes of the children nodes. For the production $A \to aA$, the length is calculated by adding 1 to the length of A. For the production $A \to b$, the length is simply 1, as b is a terminal symbol.

2.3.2 Inherited Attributes

Inherited attributes are passed from parent nodes to child nodes in the derivation tree. These attributes propagate information down and across the tree, allowing for context-sensitive analysis.

For example, consider the following grammar:

$$G = (\{S, A\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow A, A \rightarrow aA|b\}$$

We can define an inherited attribute prefix that accumulates the prefix of the generated string. The rules for computing this attribute are as follows:

$$\text{prefix}(S) = ""$$

$$\text{prefix}(A \rightarrow aA) = \text{prefix}(A) + "a"$$

$$\text{prefix}(A \rightarrow b) = \text{prefix}(A) + "b"$$

In this example, the attribute prefix is inherited because it is computed based on the attributes of the parent node. For the production $A \rightarrow aA$, the prefix is calculated by appending "a" to the prefix of A. For the production $A \rightarrow b$, the prefix is calculated by appending "b" to the prefix of A.

2.3.3 Circularity

Circularity occurs when there are cyclic dependencies between attributes, preventing their correct computation. This happens when the value of an attribute depends, directly or indirectly, on itself. Detecting and avoiding circularities is crucial to ensure the correctness and computability of attribute grammars.

Consider the following grammar:

$$G = (\{S, A\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow A, A \rightarrow aA|b\}$$

If we define the attributes such that length(A) depends on length(S), we will have a circularity:

$$\text{length}(S) = \text{length}(A)$$

$$\text{length}(A \rightarrow aA) = 1 + \text{length}(S)$$

In this example, the attribute length(S) depends on length(A), and length(A) depends on length(S), creating a cycle. This circular dependency makes it impossible to compute the values of these attributes.

To avoid circularity, we must ensure that attributes are defined in an acyclic manner. This can be achieved by carefully designing the attribute dependency graph, which represents the dependencies between attributes. The graph should be a Directed Acyclic Graph (DAG), where there are no cycles.

Detecting circularities can be complex and is often done using algorithms that analyze the dependency graph. These algorithms check for cycles and ensure that the attribute dependencies form a DAG. Ensuring acyclic dependencies is essential for the correct evaluation of attributes in attribute grammars.

By avoiding circularities, we can guarantee that attribute values can be computed correctly, enabling reliable semantic analysis and other applications of attribute grammars.

2.4 Attribute Grammars with Constraints

Attribute grammars can be extended to include constraints on attributes, which can be used to define more complex languages. In this section, we will discuss an example of an attribute grammar with two calculated attributes and permissions, utilizing attribute constraints to define a language.

2.4.1 Example Grammar

Consider the following context-free grammar G:

$$G = (\{S, A, B\}, \{a, b\}, P, S)$$

$$P = \{S \to AB, A \to aA \mid a, B \to bB \mid b\}$$

In this grammar, we will define two attributes: *length* and *permission*. The *length* attribute calculates the length of the generated string, while the *permission* attribute determines whether certain productions are allowed based on specific conditions.

2.4.2 Synthesized Attribute: Length

The *length* attribute is synthesized and calculated as follows:

$$\begin{aligned}
\text{length}(S) &= \text{length}(A) + \text{length}(B) \\
\text{length}(A \to aA) &= 1 + \text{length}(A) \\
\text{length}(A \to a) &= 1 \\
\text{length}(B \to bB) &= 1 + \text{length}(B) \\
\text{length}(B \to b) &= 1
\end{aligned}$$

2.4.3 Inherited Attribute: Permission

The *permission* attribute is inherited and determines whether a production can be applied based on the length of the string generated so far. For example, we can define a constraint

that the length of the string must be even for certain productions to be allowed:

$$\begin{aligned}
\text{permission}(S) &= \text{true} \\
\text{permission}(A \to aA) &= (\text{length}(A) \mod 2 = 0) \\
\text{permission}(A \to a) &= (\text{length}(A) \mod 2 = 0) \\
\text{permission}(B \to bB) &= (\text{length}(B) \mod 2 = 0) \\
\text{permission}(B \to b) &= (\text{length}(B) \mod 2 = 0)
\end{aligned}$$

2.4.4 Defining the Language

By using these attributes and constraints, we can define a language where the length of the generated strings must be even. This ensures that only strings with an even number of symbols are generated by the grammar.

2.4.5 Applications

Attribute grammars with constraints are useful in various applications, such as:

- **Compiler Design**: Enforcing semantic rules and type checking.
- **Language Translation**: Ensuring syntactic and semantic correctness.
- **Natural Language Processing**: Handling context-sensitive aspects of language.

By incorporating attribute constraints, we can define more precise and powerful languages, making attribute grammars a versatile tool in theoretical computer science.

2.5 Automata

Automata theory is a fundamental area of theoretical computer science that deals with the study of abstract machines and the problems they can solve. Automata are used to model computational processes and recognize patterns within input strings. Different types of automata correspond to different classes of formal languages, as described in the Chomsky hierarchy.

2.5.1 Types of Automata

2.5.1.1 Finite Automata

Finite automata (FA) are the simplest type of automata and are used to recognize regular languages. They consist of a finite number of states, transitions between these states, and an input alphabet. There are two main types of finite automata:

- **Deterministic Finite Automata (DFA)**: Each state has exactly one transition for each symbol in the input alphabet.

- **Non-deterministic Finite Automata (NFA)**: States can have zero, one, or multiple transitions for each symbol in the input alphabet.

Finite automata are used in text processing, lexical analysis, and network protocols.

2.5.1.2 Pushdown Automata

Pushdown automata (PDA) are more powerful than finite automata and can recognize context-free languages. They extend finite automata by adding a stack, which provides additional memory. This stack allows PDAs to handle nested structures, such as parentheses in arithmetic expressions or nested loops in programming languages and used in parsing and syntax analysis in compilers.

2.5.1.3 Turing Machines

Turing machines (TM) are the most powerful type of automata and can recognize recursively enumerable languages. They consist of an infinite tape, a tape head that can read and write symbols, and a set of states. Turing machines can simulate any algorithm and are used to define the limits of what can be computed and are fundamental in the study of computability and complexity theory.

2.5.1.4 Linear Bounded Automata

Linear bounded automata (LBA) are a type of Turing machine with a tape of limited length proportional to the input size. They recognize context-sensitive languages. LBAs are more powerful than PDAs but less powerful than general Turing machines, they also are used in certain types of language recognition and computational complexity studies.

2.5.2 The Relationship Between Automata and Formal Languages

The study of automata is fundamentally linked to formal languages. Each type of automaton corresponds to a class of formal languages, and understanding this relationship is key to various applications in computer science:

- **Closure Properties**: Different classes of languages exhibit specific closure properties under operations like union, intersection, and complementation. For instance, regular languages are closed under these operations, while context-free languages are not.

- **Decidability**: The ability to determine whether a given automaton accepts a particular string is a central question in automata theory. For example, the emptiness problem for finite automata is decidable, while it becomes undecidable for Turing machines.

2.5.3 Applications of Automata and Formal Languages

Automata and formal languages have numerous applications in computer science and related fields:

- **Compiler Design**: Finite automata are used in lexical analysis to tokenize input strings, while pushdown automata are used in syntax analysis.
- **Formal Verification**: Automata are used to verify the correctness of hardware and software systems.
- **Artificial Intelligence**: Automata are used in pattern recognition, natural language processing, and machine learning.
- **Network Protocols**: Automata are used to model and analyze communication protocols.

3

Grammar Tools

Understanding and applying attribute grammars can be complex, making pedagogic tools essential for effective learning and teaching. These tools provide practical, hands-on experience, helping students grasp theoretical concepts more concretely. In this chapter, we will discuss three such tools: **JFLAT**, **ANTLR**, and **CGM**. Each of these tools offers unique features and benefits that facilitate the learning process of attribute grammars.

3.1 JFLAT

JFLAT (Java Formal Languages and Automata Toolkit) is a versatile tool designed for experimenting with formal languages. It supports various computational models, including finite automata, pushdown automata, Turing machines, and different types of grammars. JFLAT provides a comprehensive environment for students to explore and understand the theoretical foundations of computer science.

3.1.1 Features

JFLAT offers several key features that enhance its utility as an educational tool:

- **Visualizing Automata**: JFLAT allows users to create and visualize finite automata, pushdown automata, and Turing machines. This visual representation helps students better understand the structure and behavior of these computational models.

- **Conversion Between Forms**: The tool supports the conversion between different forms of automata, such as converting a nondeterministic finite automaton (NFA) to a deterministic finite automaton (DFA). This feature is crucial for demonstrating the equivalence of different computational models.

- **Parsing**: JFLAT includes parsing capabilities for various types of grammars, enabling students to experiment with context-free grammars and other formal language constructs.

3.1.2 Educational Benefits

JFLAT provides significant educational benefits by offering a visual and interactive approach to learning complex concepts. The tool's ability to visualize automata and grammars helps students to:

- Gain a deeper understanding of theoretical concepts through hands-on experimentation.
- Develop problem-solving skills by working through practical exercises and projects.
- Enhance their comprehension of the relationships between different computational models.

3.1.3 Case Studies/Examples

JFLAT has been effectively used in various classroom settings to enhance the learning experience. Some examples of exercises and projects include:

- **Finite Automata Exercises**: Students can create and test finite automata to recognize specific patterns or languages, reinforcing their understanding of regular languages and automata theory.
- **Grammar Parsing Projects**: By designing and parsing context-free grammars, students can explore the syntax and structure of programming languages, gaining insights into compiler design.
- **Automata Conversion Tasks**: Assignments that involve converting NFAs to DFAs help students understand the practical applications of theoretical concepts and the importance of determinism in computation.

3.2 ANTLR

ANTLR (ANother Tool for Language Recognition) is a powerful parser generator used for reading, processing, and translating structured text or binary files. It is widely utilized in both academic and industrial settings for developing language tools and frameworks.

3.2.1 Features

ANTLR offers several notable features:

- **Parser Generation**: ANTLR can generate parsers, lexers, and tree parsers from grammars, facilitating the creation of language processors.
- **Multi-language Support**: It supports multiple programming languages, making it versatile for various development environments.

- **Building Languages and Frameworks**: ANTLR is used to build languages, interpreters, compilers, and other language-related frameworks, demonstrating its robustness and flexibility.

3.2.2 Educational Benefits

ANTLR is an excellent tool for teaching students about language parsing and compiler construction. Its practical applications help students:

- Understand the principles of syntax analysis and parsing.
- Gain hands-on experience in building parsers and compilers.
- Explore the intricacies of language design and implementation.

3.2.3 Case Studies/Examples

ANTLR has been effectively used in various educational projects and assignments. Some examples include:

- **Language Parsing Projects**: Students can create parsers for custom programming languages, enhancing their understanding of syntax and grammar rules.
- **Compiler Construction Assignments**: By building simple compilers, students learn about the different stages of compilation, from lexical analysis to code generation.
- **Framework Development**: Projects involving the development of domain-specific languages (DSLs) help students appreciate the practical applications of ANTLR in real-world scenarios.

3.3 CGM: Context-Free Grammar Manipulator

CGM is a tool developed at the University of Porto designed to assist in the manipulation and study of context-free grammars. It provides a graphical environment for editing grammars, converting them to Chomsky normal form, parsing words, and constructing parse trees interactively.

3.3.1 Features

CGM offers several key functionalities:

- **Grammar Editing**: Users can define and edit context-free grammars using a graphical interface.
- **Chomsky Normal Form Conversion**: The tool supports the conversion of grammars to Chomsky normal form, which is useful for various theoretical and practical applications.

- **Parsing and Parse Trees**: CGM allows users to parse words and visualize the corresponding parse trees, aiding in the understanding of grammar structures [**cgm**].

3.3.2 Educational Benefits

CGM aids in understanding the principles of context-free grammars through hands-on experimentation. It helps students:

- Visualize the structure and operation of context-free grammars.
- Experiment with different grammar configurations to see how changes affect parsing and parse tree construction.
- Develop a deeper understanding of the concepts of grammar rules, parsing, and syntax trees [**cgm**].

3.3.3 Case Studies/Examples

CGM has been used in various classroom activities and projects, such as:

- **Grammar Design Exercises**: Students can design grammars to recognize specific patterns or languages, reinforcing their understanding of context-free grammars.
- **Parsing Projects**: Assignments where students define grammars and parse words to observe and analyze the resulting parse trees.
- **Interactive Learning Sessions**: Instructors can use CGM in interactive sessions to demonstrate the principles of context-free grammars and engage students in hands-on learning [**cgm**].

3.4 Comparison of Tools

3.4.1 Features

Each of the three tools—JFLAT, ANTLR, and CGM—offers distinct features that cater to different aspects of teaching attribute grammars:

- **JFLAT**: Provides a comprehensive environment for experimenting with various computational models, including finite automata, pushdown automata, Turing machines, and grammars. It excels in visualizing automata and converting between different forms.
- **ANTLR**: Specializes in generating parsers, lexers, and tree parsers from grammars. It supports multiple programming languages and is widely used for building languages and frameworks.

- **CGM**: Focuses on building and testing context-free grammars. It allows users to define grammars, convert them to Chomsky normal form, and visualize parse trees.

3.4.2 Ease of Use

The ease of use varies among the tools:

- **JFLAT**: User-friendly with a graphical interface that makes it accessible for beginners. Its visualizations help in understanding complex concepts.
- **ANTLR**: Requires a steeper learning curve due to its powerful and flexible nature. It is more suitable for advanced students who have a basic understanding of language parsing.
- **CGM**: Simple and intuitive, making it easy for students to quickly start experimenting with context-free grammars. Its straightforward interface is ideal for introductory courses.

3.4.3 Educational Impact

The educational impact of each tool is significant in different ways:

- **JFLAT**: Enhances understanding of formal languages and automata theory through interactive and visual learning. It is particularly effective in demonstrating the equivalence of different computational models.
- **ANTLR**: Provides deep insights into language parsing and compiler construction. It helps students gain practical experience in building parsers and understanding the intricacies of language design.
- **CGM**: Reinforces the principles of context-free grammars through hands-on experimentation. It is effective in helping students grasp the concepts of grammar rules, parsing, and syntax trees.

3.4.4 Strengths and Weaknesses

Each tool has its strengths and weaknesses:

- **JFLAT**:
  - **Strengths**: Comprehensive, user-friendly, excellent for visual learning.
  - **Weaknesses**: May be limited in scope for advanced language parsing tasks.
- **ANTLR**:
  - **Strengths**: Powerful, flexible, supports multiple languages, ideal for advanced projects.

  - **Weaknesses**: Steeper learning curve, may be overwhelming for beginners.

- **CGM**:
  - **Strengths**: Simple, intuitive, excellent for introductory courses.
  - **Weaknesses**: Limited to context-free grammars, may not cover broader aspects of formal languages.

4

OCaml-Flat and OFLAT Systems

The OCamlFLAT library and the OFLAT platform are the main componentsof this thesis, both developed in OCaml. They are already in an advanced stage of development, thanks to the work of several students and professorswho have contributed to this project.

The OCamlFLAT library consists of various modules that make up the logical part of the system, where several FLAT theory models have already been implemented. The OFLAT platform, on the other hand, is a web application that provides the system's graphical interface.

4.1 OCamlFLAT

The OCamlFLAT library is organized into modules to ensure a clearer and more structured organization. There are modules that implement models of FLAT theory, such as regular expressions, finite automata, and context-free grammars, as well as auxiliary modules for tests, error handling, and parsers. It is important to note that the modules follow a hierarchical relationship among themselves, which facilitates the maintenance and expansion of the library.

Some of the most important modules are:

Entity: This module represents the most abstract level of the library. The entities are the exercises and the FLAT models, each containing a type, a description, and a unique ID.

Exercise: This module is responsible for supporting exercises, which are validated using UnitTests. It extends the Entity module, inheriting its properties and methods.

Model: An abstract module that also extends Entity and introduces common functionalities to the FLAT models available in the library. Examples of methods included in this module are validate, which checks the validity of a model, and accept, which processes inputs according to the model.

Examples of modules that represent FLAT models and extend Model include:

RegularExpression: This module contains all the functionalities needed to work with regular expressions, allowing the definition and manipulation of these expressions.

FiniteAutomaton: This module implements the functionalities of a finite automaton, including the definition of states and transitions.

ContextFreeGrammar: This module offers the functionalities of a context-free grammar, allowing the definition of production rules and the analysis of strings.

PushdownAutomaton: This module contains the functionalities of a pushdown automaton, which is an extension of finite automata with an additional stack for temporary storage of information.

These modules are designed to work together, forming a robust and flexible library for the implementation and analysis of FLAT theory models.

4.2 OFLAT Platform

The OFLAT platform is the graphical interface of the project, available online to the public. Its main goal is to serve as an easy-to-use educational tool. It features a static menu on the left sidebar and a dynamic workspace on the right, which changes according to the options selected on the screen. In Figure 4.1, you can see the main screen when you click on the "New model -> Finite automata" option in the sidebar.

The platform is intuitive and user-friendly. For example, to create a finite automaton, you first need to choose the option in the sidebar and then right-click on the blank workspace, where the option to create a state appears. The state can be either final or initial (Figure 4.2), and an initial state is already displayed by default. By right-clicking on the state, you can add transitions (Figure 4.3) and thus build a finite automaton. In the lower-left corner of the white screen, information about the automaton is displayed, such as whether it is deterministic, whether it is minimal, the number of useless states, the number of states, and transitions. In Figure 4.4, we can see that the created automaton is deterministic and not minimal. We can then click on the "minimize automaton" option to view a minimal version (Figure 4.5).

5

Technology

5.1 Declarative Code in OCaml

OCaml encourages a declarative programming style, where the focus is on what the program should accomplish rather than how to accomplish it. This approach aligns well with the functional programming paradigm, making the code more readable and maintainable. Declarative code in OCaml allows developers to express their intentions clearly, leading to more concise and understandable programs. By abstracting away the implementation details, developers can focus on the logic and structure of their code, which enhances productivity and reduces the likelihood of errors.

5.2 Functional Declarative Programming

Functional programming in OCaml emphasizes the use of pure functions and immutable data. This paradigm supports declarative programming by allowing developers to express computations without side effects. Pure functions, which always produce the same output for the same input and do not modify any state, are a cornerstone of functional programming. Here's a simple example:

```
(* Example of a declarative function in OCaml *)
let rec factorial n =
  if n = 0 then 1
  else n * factorial (n - 1)

(* Usage *)
let () = Printf.printf "Factorial of 5 is %d\n" (factorial 5)
```

In this example, the `factorial` function is defined declaratively, specifying what the result should be for each input without describing the steps to compute it imperatively. This approach not only makes the code easier to read and understand but also simplifies debugging and testing, as pure functions are inherently more predictable.

5.3 OCaml/JavaScript Interoperability

Interoperability between OCaml and JavaScript is made possible through the use of the `js_of_ocaml` toll, which translates OCaml code into JavaScript, allowing OCaml code to run in a browser.

This tool consists of three main components:

- `js_of_ocaml` ppx: A preprocessor that extends the OCaml library, adding functionalities that facilitate integration with JavaScript.
- `js_of_ocaml` library: A library that contains bindings for various categories of JavaScript objects, enabling OCaml code to efficiently interact with JavaScript APIs and libraries.
- `js_of_ocaml` compiler: A compiler that translates OCaml code into JavaScript, allowing OCaml programs to be executed in web-based environments.

5.3.1 JavaScript Types

Types in OCaml and JavaScript are not represented in the same way in both languages. For example, a string in OCaml is a mutable array of bytes, whereas in JavaScript, strings are constant arrays of UTF-16 code points. Therefore, it is necessary to find a way to represent JavaScript types in OCaml appropriately. The table below illustrates the correspondences between the types in the two languages.

To ensure effective interoperability, it is crucial to understand how each type is handled in OCaml and JavaScript. For instance, besides strings, other types such as numbers, arrays, and objects also have distinct representations. In OCaml, integers and floating-point numbers are treated differently compared to JavaScript, where all numbers are essentially floating-point. Arrays in OCaml are mutable, while in JavaScript, arrays can be either mutable or immutable, depending on the context.

5.3.2 Bindings

The existence and creation of bindings in `js_of_ocaml` facilitate the seamless integration of JavaScript code and libraries within the OCaml programming environment, ensuring type safety throughout the process. This powerful feature allows developers to leverage the extensive ecosystem of JavaScript while maintaining the robust type-checking capabilities of OCaml.

Although `js_of_ocaml` already includes a plethora of useful bindings, developers are not limited to these pre-existing options. It remains entirely feasible to create new bindings tailored to specific project needs, thereby enabling interaction with a wide array of additional JavaScript libraries and objects. This flexibility is particularly advantageous for projects that require the integration of specialized JavaScript functionalities or the utilization of cutting-edge libraries that may not yet be covered by the existing bindings.

By creating custom bindings, developers can ensure that their OCaml applications can interact with the latest JavaScript technologies, thereby enhancing the functionality and versatility of their projects. This approach broadens the scope of what can be achieved with `js_of_ocaml` and empowers developers to innovate and adapt to the ever-evolving landscape of web development

5.3.3 `js_of_ocaml` Modules

The `js_of_ocaml` library already includes several modules with predefined bindings, but only a few are necessary for the development of the OFLAT platform.

Below are some examples of these modules and submodules:

- Js Module: This module contains definitions of JavaScript types, functions for standard JavaScript objects, and conversion functions between OCaml and JavaScript types. It is essential for ensuring interoperability between the two languages.
- Js.Unsafe Submodule: This submodule allows for operations on JavaScript that are not safe, offering greater flexibility but requiring extra caution to avoid errors and vulnerabilities.
- `Dom_html` Module: Defines and manipulates HTML DOM elements, enabling the creation and modification of HTML structures directly from OCaml code. This module is crucial for building the graphical interface of the platform.
- Firebug Module: Allows for the visualization and debugging of objects in the browser's developer console, facilitating the development and debugging process.

5.4 Representation in OCaml

When working with OCaml and JavaScript interoperability, understanding how to represent various attributes is crucial. OCaml uses synthesized and inherited attributes to manage data and behavior in a structured way. These attributes play a significant role in the language's type system and its ability to enforce strong type safety.

5.4.1 Synthesized Attributes

These are attributes computed from the values of other attributes. For example, in a syntax tree, the type of an expression might be synthesized from the types of its sub-expressions. Synthesized attributes help in propagating information upwards in a hierarchical structure, ensuring that each node has the necessary context to perform its operations.

5.4.2 Inherited Attributes

These are attributes passed down from parent nodes to child nodes in a structure. For instance, the scope information in a syntax tree can be inherited from parent nodes to

ensure proper variable resolution. Inherited attributes facilitate the flow of information downwards, providing child nodes with the context they need to function correctly.

5.4.3 Rules

The rules define how attributes are calculated. They consist of a head, which specifies the attribute being computed, and a body, which describes the computation. These rules are essential for maintaining consistency and correctness in the representation of data.

Here's a simple example to illustrate these concepts:

```
(* Example of synthesized and inherited attributes in OCaml *)
type expr =
  | Const of int
  | Add of expr * expr

(* Function to evaluate an expression with synthesized attributes *)
let rec eval = function
  | Const n -> n
  | Add (e1, e2) -> eval e1 + eval e2

(* Usage *)
let () = Printf.printf "Result of 3 + 4 is %d\n" (eval (Add (Const 3, Const 4)))
```

In this example, the `eval` function synthesizes the result of an addition expression from its sub-expressions. This approach ensures that each part of the expression is evaluated in a consistent and predictable manner, leveraging the power of OCaml's type system to enforce correctness.

5.5 Cytoscape

Cytoscape is an open-source software platform that, although initially designed for biological research, is now a general tool for graph visualization and analysis. In this project, the OFLAT platform is used to visualize the various models supported by the application. To use this library, it is necessary to create bindings for OCaml/JavaScript interoperability.

6

Lorem Ipsum